



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE INGENIERÍA

INGENIERIA EN SISTEMAS INTELIGENTES

Interfaces Gráficas con Aplicaciones

Reporte: JUEGO UNO

Alumnos:

Iker Alejandro Arriaga López

María Fernanda Hernández Galván

Regina Escobar Zárate

San Luis Potosí, 09 de noviembre del 2026

DESCRIPCIÓN GENERAL DEL PROYECTO

Este proyecto consiste en la creación de una adaptación interactiva del juego de mesa "UNO". El desarrollo de este entorno tiene una interfaz gráfica intuitiva y familiar para el usuario, está diseñado para 3 jugadores en una modalidad de multijugador local, también cuenta con una base de datos sencilla que nos permite la gestión y persistencia de datos como la información de los jugadores, el historial de partidas perdidas o ganadas por cada jugador y todos los movimientos hechos en cada turno.

Todo el sistema está desarrollado en Visual Studio con el marco GUI Windows Forms, con una base de datos modelada en MySQL Workbench.

ARQUITECTURA DEL PROYECTO

▪ Patrón Arquitectónico

- **Tipo de arquitectura:** Arquitectura de Tres Capas/Cliente-Servidor desacoplada
- **Descripción general:** El sistema separa la lógica de la interfaz de usuario, la lógica de negocio y el almacenamiento de datos en componentes independientes que se comunican entre sí mediante servicios web (API) y conectores directos a bases de datos relacionales.

▪ Descripción de las Capas

- **Capa de Presentación y Lógica del Cliente:** Se usó C# (.NET Framework /.NET Core), Windows Forms. Con está tecnología se gestiona la interfaz gráfica del usuario (el tablero, las cartas, los turnos, las "animaciones" y las ventanas de diálogo como el Form3). Contiene las reglas del juego de mesa de UNO (validación de jugadas, efectos de cartas especiales y control de tres jugadores).
- **Capa de Servicios/ API (Backend Intermedio-Python):** Python y FastAPI/Uvicorn se utilizan actuando como un puente de servicios web (API REST). Expone endpoints HTTP (por ejemplo, /jugadores) para procesar peticiones y entregar información estructurada en formato **JSON** al cliente de escritorio cuando se le solicita el ranking de ganadores.

- **Capa de Datos (Almacenamiento-MySQL):** Las tecnologías usadas fueron: MySQL Server, conector MySqlConnector y mysql.connector.
  Almacena de manera persistente la información de los usuarios (jugador), las sesiones activas y finalizadas (partida), las estadísticas acumuladas de victorias y derrotas (historial_partidas) y el registro detallado de las acciones de los usuarios (log_juego).

- **Flujo de Datos y Comunicación**

  - **Interacción local:** El usuario realiza una acción en el juego (Form1), el cual interactúa directamente con la base de datos MySQL mediante consultas SQL (INSERT/UPDATE) usando MySqlConnector para guardar logs y registrar estados de la partida en tiempo real.
  - **Consulta externa vía API:** Al finalizar la partida, el juego abre la ventana de resumen (Form3), la cual realiza una petición HTTP asíncrona (GET) hacia el servidor local de la API en Python.
  - **Respuesta:** La API consulta los datos en MySQL, los serializa a formato JSON y los devuelve al cliente de C# para mostrarlos formateados en pantalla.

- **Descripción de la API**

  La comunicación entre el cliente (C#), y el servidor (FastApi) se rige bajo los principios de una API REST, utilizando el formato estándar JSON para el intercambio de datos. El manejo de la persistencia se realiza mediante la ejecución directa de consultas en la base de datos MySQL gestionadas desde cada endpoind de la API

- **Peticiones POST (Modificación e Inserción de Datos):** Se emplean exclusivamente para todas aquellas acciones que implican registrar, crear o alterar información dentro de la base de datos MySQL. Los parámetros necesarios para estas operaciones no viajan expuestos en la URL, sino que se envían de forma estructurada dentro del cuerpo de la petición (*request body*) en formato JSON.

- **Peticiones GET (Consulta de Información):** Se utiliza únicamente para la lectura y recuperación de datos desde el servidor hacia el cliente, sin alterar ni modificar el estado del sistema.

A continuación, se detalla formalmente cada uno de los endpoints implementados en el servidor junto con sus respectivas operaciones de consulta en la base de datos:

1. **Registro o Recuperación de Jugadores**

    - **Ruta y Método:** POST , **/jugadores/registrar**
    - **Propósito:** Gestionar el alta y validación de los participantes al iniciar una sesión de juego.
    - **Detalles Técnicos y Queries:** Recibe un objeto JSON con el nombre del usuario. Ejecuta una *query* de búsqueda (SELECT id_jugador FROM jugador WHERE nombre = %s) para verificar si el registro ya existe; de ser así, recupera el identificador existente (es_nuevo = False). Si el usuario no existe, ejecuta una sentencia de inserción (INSERT INTO jugador (nombre)...) y un segundo comando para inicializar sus estadísticas en la tabla historial_partidas con valores en cero, retornando el nuevo ID junto con es_nuevo = True.

2. **Almacenamiento de Logs de Juego**

    - **Ruta y Método:** POST, **/logs**
    - **Propósito:** Mantener un registro histórico detallado de los movimientos y eventos ocurridos durante el desarrollo de cada partida.
    - **Detalles Técnicos y Queries:** Recibe el identificador de la partida, el del jugador y la descripción del movimiento. Ejecuta una *query* de inserción (INSERT INTO log_juego (id_partida, id_jugador, movimiento, fecha_hora) VALUES (%s, %s, %s, NOW())), incorporando de manera automática la marca de tiempo actual del servidor.

3. **Creación de Nueva Partida**

    - **Ruta y Método:** POST, **/partidas/crear**
    - **Propósito:** Inicializar una nueva sesión de juego en el sistema de base de datos.
    - **Detalles Técnicos y Queries:** Ejecuta una sentencia de inserción (INSERT INTO partida (estado) VALUES ('En curso')) para registrar una nueva sesión. Al completarse, extrae y devuelve el identificador autoincremental generado (id_partida) para que la aplicación cliente lo utilice en el control de turnos y acciones.

4. **Actualización de Historial de Partidas**

    - **Ruta y Método:** POST, **/historial/actualizar**
    - **Propósito:** Actualizar las estadísticas de rendimiento (victorias o derrotas) de los participantes al concluir un juego.
    - **Detalles Técnicos y Queries:** Recibe el identificador del jugador y un valor booleano (gano). Según el parámetro recibido, la API ejecuta una *query* de

actualización (UPDATE historial_partidas SET ganadas = ganadas + 1 ... o su equivalente para perdidas) incrementando en uno el contador correspondiente en la base de datos.

5. **Consulta del Ranking Global de Jugadores**

- **Ruta y Método:** GET /jugadores
- **Propósito:** Proveer la información consolidada para la pantalla de resumen y estadísticas finales del cliente (Form3).
- **Detalles Técnicos y Queries:** Ejecuta una consulta multitabla optimizada (SELECT j.id_jugador, j.nombre, h.ganadas AS partidas_ganadas FROM jugador j JOIN historial_partidas h ON j.id_jugador = h.id_jugador) para relacionar a los usuarios con sus respectivas estadísticas. Retorna una lista estructurada en formato JSON con el ranking global de jugador

▪ Stack Tecnológico

- **Lenguajes de programación, de consulta y manipulación de datos:** C#, Python, SQL.
- **Frameworks y Librerías:** Windows Forms (UI), FastAPI/Uvicorn (API), MySqlConnector & Newtonsoft.Json (Conectividad y serialización).
- **Base de datos:** MySQL
- **Control de versiones:** Git y GitHub (bajo un flujo de trabajo basado en ramas y Pull Request).

DIVISIÓN DE TAREAS

Integrante encargado	Descripción de la tarea
Regina Escobar Zarate / regin4-git	- **Creación de la base de datos y diagrama entidad-relación.** - **Creación de la clase Jugador con atributos y métodos:** La clase jugador se creará con el propósito de tener a los jugadores en la lógica del juego como objetos pudiendo usar sus métodos RecibirCarta(), JugarCarta() y ContarCartas() haciendo más fácil el desarrollo de las jugadas. - **Lógica de los turnos:** Creación de las funciones ActualizaTurnoLabel() y AvanzarTurno(), dichas funciones permitirán llevar un control de los turnos de los 3 jugadores en un ciclo de reinicio cuando se llegue al turno inválido como 4 o -1 dependiendo de la dirección del juego. - **Lógica de las jugadas (Tirada de cartas):** Realizó un evento (CartaClick()) para el momento de hacer click en cualquier carta (se suscriben las cartas al evento cuando se reparten al jugador, cuando este roba del mazo o cuando come un +4 o un +2). Dicho evento validará turno y que sea una jugada válida (mismo color, mismo número, etc). Creación de la función RealizaJugada() que removerá la carta del jugador tanto del tablero como de la lógica y AplicarEfectoCarta() que aplicará los diferentes efectos de las cartas +4, +2, bloqueo, reversa, además de la función DarCartasASiguienteJugador() para cuando alguien tire un +4 o un +2. - **Implementación de ventanas para caso COMODÍN**: Diseño de una pequeña ventana que permite elegir un color de los 4

	disponibles para cuando se haya tirado un comodín de color o un +4. - **Implementación del robo desde mazo**: Creación de la función RobarMazoCarta() que permite tomar una carta cuando no tienes cartas válidas. Creación de la función JugadaEstrategica() para cuando se robó una carta del mazo y es válida se muestra una ventana para preguntar al jugador si desea jugarla o desea conservarla. - **Reinicio Juego:** Reinicia el juego limpiando los controles de la interfaz y la pila de cartas, también las variables las reinicia (direccionJuego, turnoActual, etc), vuelve a barajar el mazo y a repartir cartas entre jugadores.
	- **Implementación de la clase Cliente Juego(1ra versión):** Implementar la versión base de esta clase inicia el socket del cliente, crea las funciones para conectar al cliente con el servidor dependiendo de que cliente sea (Jugador 1, 2 o 3), crea la función para poder enviar una acción al servidor de acuerdo con lo que hizo el cliente (tiró una carta, robo una carta, etc), implementación de la función de escucha al servidor para poder actualizar la interfaz con las acciones que hagan los otros jugadores y sus clientes hayan enviado al servidor, procesamiento de la orden del servidor usando switch se obtiene la orden específica que se enviará al servidor (conexión/desconexión, efecto de carta, evento del botón UNO, reinicio, reparto de mano, robo de carta, tirada de carta y comodin). - **Hacer la conexión básica con el Servidor:** Implementar la versión base de la clase Servidor para que se pueda hacer la definición de su dirección IP, pueda iniciar su socket, iniciar la partida(ir registrando a los 3 clientes) y escuchar a los diversos clientes. Nota: Versión prueba para enviar una sola carta.

	- **Paso de lógica del juego al Servidor:** Inicia las modificaciones para que el servidor pueda ejecutar la lógica desde su misma implementación e ir "eliminando" dichas funciones de LógicaUno. Hacer que el servidor pueda barajar el mazo de cartas y hacer la repartición a cada uno de sus clientes, pueda hacer las validaciones correctas de cada jugada de cada cliente (color o número correcto, comodines o +4/+2), pueda penalizar a los clientes de ser necesario; para esto el servidor debe escuchar al cliente y validar cualquier petición(Conexión, Desconexión, botón UNO, etc). - **Modificar funciones de Lógica del juego:** Ir cambiando las funciones que ya no son necesarias(fueron agregadas al Servidor) para la lógica de las jugadas. Ir borrando el contenido (solo el contenido, no la función) de las funciones en LógicaUno que ya son validadas totalmente en el servidor y las demás se irán comunicando con este y validando en la misma.
Iker Alejandro Arriaga López	- **Implementación de la interfaz de juego:** La interfaz conlleva la distribución y diseño de los controles dentro de la ventana y su integración en la lógica del juego; cada jugador debe poseer un numero de cartas y estas deben verse reflejadas en el tablero; al igual que el tablero debe mostrar las cartas jugadas. La interfaz se separará en 4 secciones distintas: 1. Jugador 1, 2 y 3: Se encargará de mostrar las cartas y nombre correspondientes a cada jugador. 2. Tablero: En esta sección se dibujarán las cartas apiladas y se irán actualizando según vaya transcurriendo el juego. También se agregará un botones para reiniciar el juego y para gritar "¡UNO!". Gran parte del código correspondiente a esta implementación se encontrará en la ventana principal (Form1).

	- **Creación de la clase Carta:** La clase Carta cumple con la función de representar una carta de juego dentro del código para manipular de forma más eficiente tanto los elementos gráficos que la describen, como su manejo en las reglas internas del juego. Para explicarme mejor, cada carta contendrá un enumerador, "TipoCarta", que determina el tipo de carta que es: - NUMERO: La carta contiene un numero entre 0 y 9. - BLOQUEA: Bloquea el turno del siguiente oponente. - REVERSA: Cambia el sentido del juego. - MASDOS/MASCUATRO: Obliga a robar al jugador 2/4 cartas y también le hace perder su turno. - COMODIN: Permite cambiar el color actual del juego. También se agregará otro enumerador "ColorCarta", que determinará el color de la carta, y en caso de no tenerlo lo generaliza a un valor "ESPECIAL". - **Creación de la clase Baraja:** Ya explicado el porqué de la clase Carta, esta clase, Baraja, se encargará de definir cómo usar a Carta dentro de las reglas del juego. Según las reglas oficiales de UNO: - Primero, solo existen dos ejemplares de un mismo número para cada color, a excepción del número 0, que solo cuenta con uno en cada color. - Segundo, las cartas especiales de colores (bloquear, comer dos, cambiar sentido) también cuentan con dos ejemplares para cada color. - Y tercero, solo hay 4 ejemplares para las cartas de comodín y comer cuatro. La clase Baraja se encargará entonces de crear este número concreto de cartas, además de barajear este mazo, y asegurarse de quitar cartas al comer.

	- **Separación de responsabilidades de Form1:** Anteriormente, Form1 se encargaba de: o Dibujar los controles. o La lógica del juego. o Hacer las llamadas a la API. Esto funciona, pero se necesita separar estas responsabilidades para una mejor organización del código y escalar el programa de forma eficiente. El plan será dividir esencialmente el código de Form1 en tres clases concretas: o **MesaUNO:** Se encargará de dibujar los controles del juego, pero también necesitará conectarse a LogicaUNO para llevar a cabo dichos cambios. o **LogicaUNO:** Contendrá todo lo referente a la lógica del juego UNO, manejará las cartas, los jugadores, y validará las jugadas y las victorias. También se hará las llamadas a la base de datos con LlamadaBD. o **LlamadaBD:** Como lo dice su nombre, únicamente realizara llamadas a la base de datos con los datos recibidos de LogicaUNO.
Maria Fernanda Hernandez Galvan / f3rhg/ Fernanda	- **Conexión inicial a la base de datos:** Para la conexión inicial a la base de datos se utilizará el código visto en clase, únicamente se verificará que la conexión se haga con éxito, es decir, se implementará y ejecutará un mensaje en pantalla que indique de manera clara que la conexión se estableció correctamente. - **Implementación de registro de jugadores y conexión a la base de datos:** Para la implementación del registro, se planea diseñar una interfaz/ventana en la cual los jugadores se puedan registrar únicamente con su nombre de usuario. Para llevar a cabo estas acciones, la lógica correspondiente deberá ser implementada en

	el código de la conexión inicial a la base de datos, considerando los siguientes escenarios: 1. Cuando un jugador acceda al juego por primera vez, se mostrará un mensaje indicándole que es "Nuevo usuario". 2. Si el jugador ya ha sido registrado, se deberá verificar su existencia en la base de datos y se mostrará un mensaje como recordatorio de que ya estaba registrado como usuario. - **Implementación de la API** Para la implementación correcta de la API, se tendrá que seguir una serie de pasos: 1. Lo primero que se debe hacer es establecer la estructura que tendrá nuestra API. Al igual que en la conexión a la base de datos, se implementará utilizando como base el código visto en clase. En este caso se deberá implementar usando el lenguaje Python. 2. Una vez establecida la estructura, se deberá conectar la API a nuestra base de datos utilizando la librería **mysql.connector,** permitiendo crear endpoints (como /jugadores) que ejecuten consultas SQL para extraer y retornar la información de los usuarios en formato JSON. 3. En el código del juego se creará la clase "ApiService" que realizará la petición GET a un endpoint determinado; se implementará utilizando HttpClient debido a que nos pareció la mejor opción de las que se nos presentaron en clase. - **Implementación de la ventana "Resumen de partida"**: Esta ventana se implementará con el fin de verificar que la API se conecte correctamente a la base de datos para obtener la información en formato JSON. En esta sección del código (en el juego) es donde se ejecutará la petición GET al servidor para traer el JSON, darle un formato legible e imprimirlo en pantalla. Si la petición se realiza con éxito los datos se mostrarán en un ranking ordenando a los jugadores según sus partidas ganadas, si falla mostrará un mensaje de error.

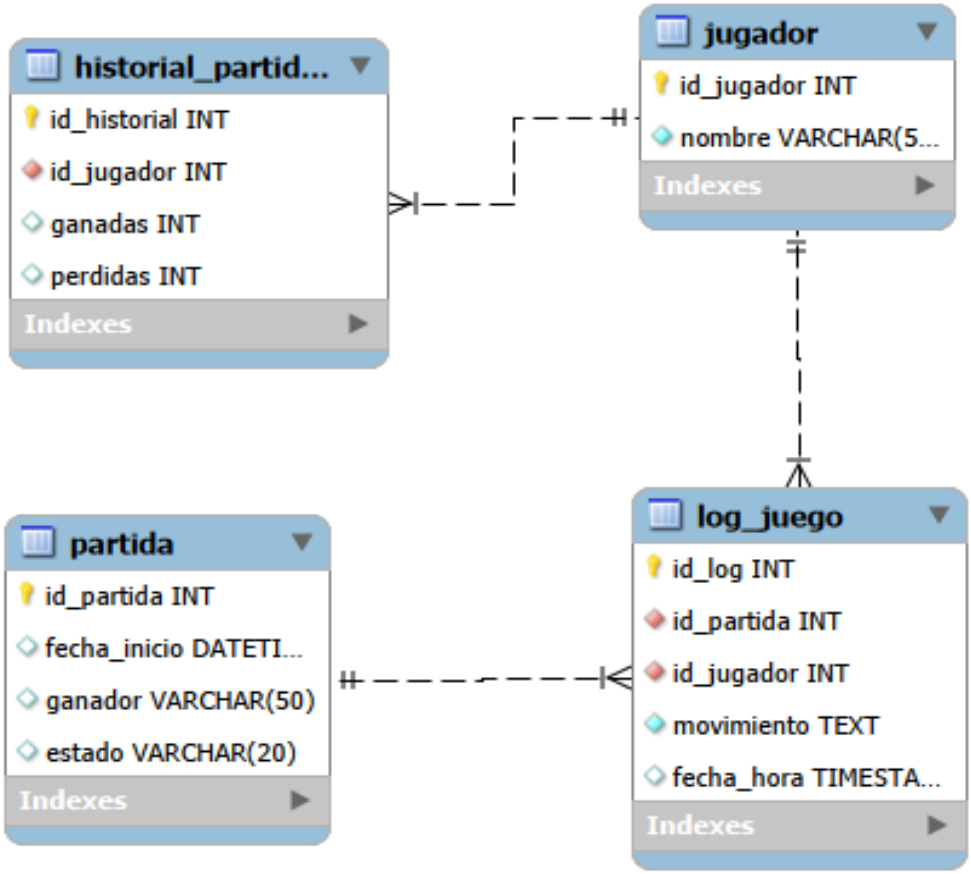
	- **Implementación de la lógica para verificar que un jugador haya ganado**: Para que un jugador haya ganado deberá haber tirado su última carta y haber dicho “UNO”. - **Implementación la lógica para la penalización**: En caso de que un jugador tire su última carta si haber dicho “UNO”, se le penalizará. Esta penalización consistirá en sumarle dos cartas a su “mano”

Tareas asignadas para la implementación del modelo Cliente-Servidor

Integrante encargado	Descripción de la tarea
Regina Escobar Zarate / regin4-git	**Implementación de la comunicación cliente-servidor y adaptación de la lógica del juego:** • Implementar la comunicación mediante sockets TCP para permitir que los clientes envíen sus acciones al servidor. • Modificar ClienteJuego para enviar las acciones realizadas por los jugadores, como tirar cartas, robar del mazo y utilizar cartas especiales. • Trasladar la lógica principal de LogicaUNO a ServidorUNO, de manera que el servidor sea el encargado de validar las jugadas, controlar los turnos y aplicar los efectos de las cartas. • Modificar las funciones de LogicaUNO para que actualicen el estado local de la partida a partir de la información recibida del servidor. • Adaptar MesaUNO para recibir y mostrar los cambios realizados por los demás jugadores, manteniendo sincronizada la interfaz gráfica.
Iker Alejandro Arriaga López	**Modificación de la arquitectura del juego para implementar el modelo cliente-servidor:** • Reestructurar la arquitectura original del juego UNO para permitir la implementación del modelo cliente-servidor. • Separar las responsabilidades de la interfaz gráfica, la lógica del juego y la comunicación con la base de datos.

	• Adaptar la estructura del proyecto para que MesaUNO se encargue de la interfaz gráfica, LogicaUNO del estado local de la partida y LlamadaBD de la comunicación con la API. • Preparar la arquitectura para trasladar el control principal de las partidas a ServidorUNO, permitiendo que tres jugadores participen desde computadoras diferentes.
Maria Fernanda Hernandez Galvan / f3rhg/ Fernanda	**Integración y corrección de la comunicación cliente-servidor, optimización de la interfaz y conexión con la API:** • Integrar las modificaciones realizadas en el cliente y el servidor para permitir que tres jugadores participen en una misma partida desde computadoras diferentes. • Modificar la comunicación con FastAPI para centralizar el registro de jugadores, partidas, movimientos y resultados en una sola base de datos MySQL. • Configurar la API para permitir conexiones desde las computadoras secundarias mediante la dirección IP de la computadora principal. • Implementar y corregir las funciones del servidor encargadas de registrar las partidas, guardar los movimientos y actualizar los resultados mediante peticiones HTTP a la API. • Corregir los problemas de conexión TCP entre ServidorUNO y ClienteJuego, asegurando la recepción y el procesamiento de los mensajes. • Optimizar la actualización de los controles de `MesaUNO` para evitar que las cartas y otros elementos gráficos se redibujen constantemente después de cada jugada. • Modificar el funcionamiento del reinicio de partidas para permitir que los jugadores comiencen una nueva ronda sin cerrar las ventanas ni perder la conexión con el servidor. • Realizar las pruebas finales de integración para verificar el funcionamiento del juego en las tres computadoras.

DIAGRAMA ENTIDAD-RELACIÓN



LISTADO DE COMMITS

NOVIEMBRE

Commit / Referencias	Fecha y Hora	Autor	Descripción
1cd1391	Fri Nov 6 11:04:02 2026	f3rhg	correccion de errores en MesaUNO
7f9d287	Fri Nov 6 11:02:01 2026	f3rhg	Modificacion reinicio de partida
30ed2e7	Fri Nov 6 08:43:21 2026	f3rhg	Correcciones en MesaUNO para corregir el refresh de la interfaz

197756e	Thu Nov 5 21:53:27 2026	f3rhg	Modificaciones en el servidor para corregir la conexion con el cliente
c881f86	Thu Nov 5 21:51:29 2026	f3rhg	Modificaciones en los archivos para la implementación final de la conexion cliente-servidor
8c092d1	Wed Nov 4 09:58:29 2026	Maria Fernanda Hernandez Galvan	Merge pull request #27 from Ale-Pez/feature/ComunicacionClienteServidor - Feature/comunicación cliente servidor
62893e1	Wed Nov 4 09:57:04 2026	regina4-git	Modificaciones de MesaUno
2563d0d	Wed Nov 4 09:42:47 2026	regina4-git	Cambio de logica del juego de LogicaUno a ServidorUno
c0de74f	Wed Nov 4 08:30:02 2026	Maria Fernanda Hernandez Galvan	Merge pull request #26 from Ale-Pez/feature/ComunicacionClienteServidor - Conexion al Servidor/ Envio de Accion de Carta
65fb9ba	Wed Nov 4 08:28:32 2026	regina4-git	Conexion al Servidor/ Envio de Accion de Carta
4e55db7	Wed Nov 4 05:12:56 2026	Iker Alejandro Arriaga López	Merge pull request #25 from Ale-Pez/servidor - Implementar baraja compartida, reparto de cartas y control inicial de turnos en servidorUNO
16076fb	Sun Nov 1	f3rhg	implementar recepcion de cartas y turnos en el cliente desde el servidor

	16:14:53 2026		
c7c6692	Sun Nov 1 15:49:14 2026	f3rhg	Implementar baraja compartida, reparto de cartas y control inicial de turnos en ServidorUNO
0345a0b	Sun Nov 1 15:33:20 2026	Maria Fernanda Hernandez Galvan	Merge pull request #24 from Ale-Pez/feature/ClienteApi - Implementación de la clase Cliente Juego

OCTUBRE

Commit / Referencias	Fecha y Hora	Autor	Descripción
515fb22	Fri Oct 30 11:58:38 2026	regina4-git	Implementación de la clase Cliente Juego
e6d11dd	Mon Oct 26 11:39:14 2026	Maria Fernanda Hernandez Galvan	Merge pull request #23 from Ale-Pez/reestructura_interfaz - Reestructura del programa
da1d5bd	Sun Oct 18 17:27:40 2026	Iker Alejandro Arriaga López	Separación de responsabilidades completa, MesaUNO se encarga de la interfaz gráfica, LogicaUNO de aplicar los cambios internos del juego y LlamadaBD de conectarse con la API
a4d8ca4	Thu Oct 15 17:10:28 2026	Iker Alejandro Arriaga López	Agrega las clases de reestructuración del código: LlamadaBD, LogicaUNO y MesaUNO
bd8bdb7	Thu Oct 15	Iker Alejandro	Primer avance de la reestructuración del código: Separa de Form1 la logica del juego, las llamadas a la API y el manejo de la interfaz; cada uno en su propia clase

	17:08:59 2026	Arriaga López	(LogicaUNO, LlamadaBD, MenuUNO respectivamente)
4650bfa	Wed Oct 14 21:58:13 2026	Iker Alejandro Arriaga López	Actualizacion del Reporte
8e0d7bd	Tue Oct 13 16:38:25 2026	f3rhg	Modificacion de la arquitectura y funcionamiento de la API
e350bdd	Sat Oct 10 21:54:21 2026	f3rhg	Merge branch 'main' of https://github.com/Ale-Pez/UNO-Project into API
7687fe4	Sat Oct 10 21:51:07 2026	f3rhg	cambios en forms3.cs
2fb1afa	Sat Oct 10 18:34:47 2026	Regina Escobar Zarate	Merge pull request #17 from Ale-Pez/mejoras_interfaz - Mejoras interfaz
bea426b	Sat Oct 10 18:26:18 2026	Iker Alejandro Arriaga López	Soluciona error al cambiar de color con una carta numerica. Quita la figura de referencia para la figura de colores
c328ad6	Sat Oct 10 18:08:14 2026	Iker Alejandro Arriaga López	Agrega una figura para indicar el color de carta del juego actual
153416f	Sat Oct 10 16:40:17 2026	Regina Escobar Zarate	Merge pull request #16 from Ale-Pez/mejoras_interfaz - Mejoras interfaz

c30a7a6	Sat Oct 10 12:43:29 2026	Iker Alejandro Arriaga López	Recoloca la etiqueta que emerge al jugar ciertas cartas (ej. +4)
f35e8b2	Thu Oct 8 11:49:40 2026	Iker Alejandro Arriaga López	Los controles del centro (Mazo, Pila, etc.) Han sido centrados. Tambien se han hecho ligeras correcciones de ortografía o código muerto
442d602	Wed Oct 7 19:13:48 2026	Maria Fernanda Hernandez Galvan	Merge pull request #15 from Ale-Pez/feature/CambiosMensajesSpam - Feature/cambios mensajes spam
ff0949e	Wed Oct 7 19:02:12 2026	regina4-git	Ignorar carpeta __pycache__ en .gitignore
810fac3	Wed Oct 7 18:50:28 2026	regina4-git	Modificacion mensajes Spam a Label con timer
e6a35a5	Wed Oct 7 17:00:27 2026	Regina Escobar Zarate	Merge pull request #14 from Ale-Pez/API - Modificacion de la interfaz "Registro"
7958065	Tue Oct 6 11:53:11 2026	f3rhg	Modificacion de la interfaz "Registro"
d2e6486	Mon Oct 5 06:29:04 2026	Iker Alejandro Arriaga López	Merge pull request #13 from Ale-Pez/feature/ReinicioJuego-BarajeoPilaCartas - Reinicio del juego y barajar la pila de cartas
ccc93e5	Sun Oct 4 20:31:12 2026	regina4-git	Reinicio de juego en Form3

0eddbde	Sun Oct 4 20:00:34 2026	regina4-git	Reinicio de juego en Form1
ab84444	Sun Oct 4 19:08:25 2026	regina4-git	Barajar pila de cartas
782743b	Sun Oct 4 18:18:47 2026	Regina Escobar Zarate	Merge pull request #12 from Ale-Pez/API - Se adjuntó el archivo que contiene la API al proyecto
00176b3	Sun Oct 4 14:37:14 2026	f3rhg	Se adjuntó el archivo que contiene la API al proyecto
74a7330	Sun Oct 4 14:31:45 2026	Iker Alejandro Arriaga López	Merge pull request #11 from Ale-Pez/API - Ajustes en la ventana de "resumen de partida" y conexión de la API a C#
3f0d984	Sun Oct 4 14:27:55 2026	f3rhg	Ajustes en la ventana de "resumen de partida" y conexion de la API a c#
21c2c39	Sun Oct 4 12:44:41 2026	Maria Fernanda Hernandez Galvan	Merge pull request #10 from Ale-Pez/feature/cambio-002-bd - Segunda actualizacion de la BD, el juego ya guarda los datos en la BD
b05e63a	Sun Oct 4 12:38:57 2026	regina4-git	Segunda actualizacion de la BD, el juego ya guarda los datos en la BD
0593804	Sun Oct 4 10:59:32 2026	Iker Alejandro Arriaga López	Implementación de la conexión a la API para conectar con la base de datos
205d75f	Sun Oct 4 03:15:59 2026	f3rhg	correciones en la logica de penalizacion

7799c7b	Sun Oct 4 02:44:28 2026	f3rhg	Implementacion de la penalizacion (si el jugador no dice UNO antes de tirar su ultima carta se agregan dos cartas a su panel)
292ca7a	Sat Oct 3 23:31:02 2026	f3rhg	implementacion del metodo "verificar ganador"
6d69b09	Sat Oct 3 22:52:46 2026	f3rhg	Implementacion de la interfaz "resumen de partida" que obtiene el formato json de la API
b1102da	Sat Oct 3 21:41:49 2026	f3rhg	Creacion de la clase apiService (puente de comunicación entre C# y la api)
8870af2	Sat Oct 3 20:39:14 2026	Regina Escobar Zarate	Merge pull request #8 from Ale-Pez/feature/boton_uno - Correción de detalles de la interfaz
e9637b4	Sat Oct 3 20:25:31 2026	Iker Alejandro Arriaga López	Comprobación de que la primer carta de la pila sea únicamente numérica
6c30212	Sat Oct 3 20:19:26 2026	Iker Alejandro Arriaga López	Agrega el boton de UNO y lo oculta al iniciar la ventana Form1
6986b43	Sat Oct 3 19:45:08 2026	f3rhg	Merge pull request #7 from Ale-Pez/feature/TiradaDeCartas-AgregarALogJuego - Feature/tirada de cartas agregar a log juego
6dbac51	Sat Oct 3 19:22:13 2026	regina4-git	Actualizar log-juego y registrar-jugador
49804ea	Sat Oct 3 18:41:11 2026	regina4-git	Funcion para escoger color del comodin y robar del mazo restante

850612d	Sat Oct 3 16:02:14 2026	regina4-git	Evento tira carta, Funcion valida turno y Aplicar efecto de carta
5b3f793	Sat Oct 3 14:16:01 2026	regina4-git	Verificar Jugada Valida y Logica de turnos: Avanzar y Actualizar
f89a99c	Sat Oct 3 11:59:40 2026	Iker Alejandro Arriaga López	Merge pull request #6 from Ale-Pez/feature/cambio-001-bd - Base de datos corregida en historial_partida y log_juego
6c2cacd	Sat Oct 3 11:44:53 2026	regina4-git	Base de datos corregida en historial_partida y log_juego
30d638f	Thu Oct 1 11:10:24 2026	Iker Alejandro Arriaga López	Merge pull request #5 from Ale-Pez/ConexionBD - Conexion BD
b8553f7	Thu Oct 1 11:07:24 2026	Iker Alejandro Arriaga López	Merge pull request #4 from Ale-Pez/feature/clase-Jugador - Implementacion de la clase Jugador y actualizacion de Repartir_Cartas

SEPTIEMBRE

Commit / Referencias	Fecha	Autor	Descripción
a19e7cc	Wed Sep 30 17:29:31 2026	Fernanda	Implementacion de registro de jugadores y base de datos
d2f706d	Wed Sep 30 12:53:18 2026	Fernanda	Merge remote-tracking branch 'origin/feature/clase-Jugador' into ConexionBD

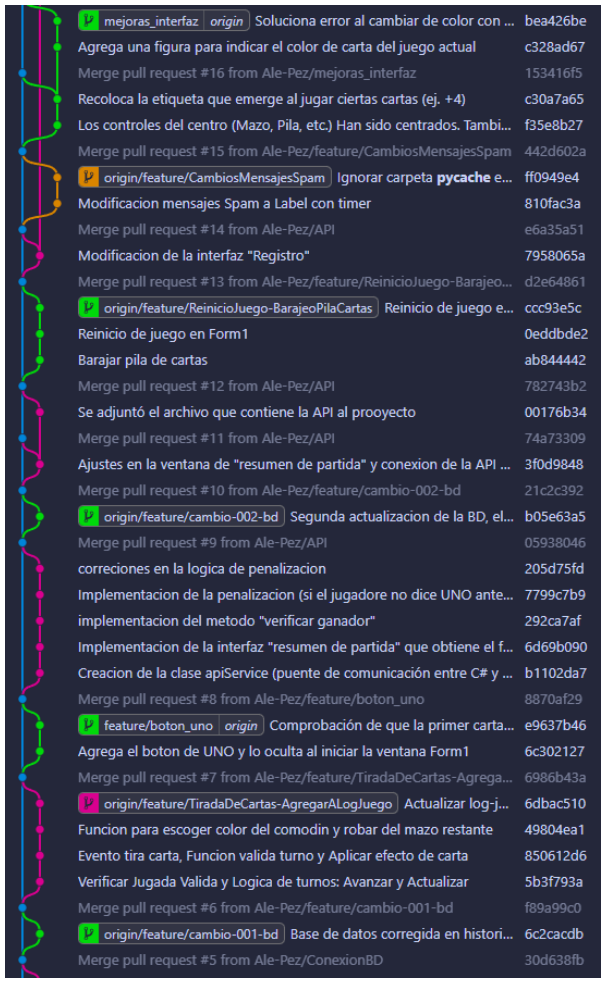
3ba2eba	Mon Sep 28 19:50:33 2026	Regina	Implementacion de la clase Jugador y actualizacion de Repartir_Cartas
75d1031	Sun Sep 27 17:44:49 2026	Iker Alejandro Arriaga López	Merge pull request #3 from Ale-Pez/interfaz_nueva - Mejoras en la interfaz
829852a	Sun Sep 27 17:41:38 2026	Iker Alejandro Arriaga López	Enseña las cartas correspondientes a cada jugador y la pila
26d5c62	Sun Sep 27 16:54:30 2026	Iker Alejandro Arriaga López	Implementa logica bajo la pila de cartas
16ca7c9	Sun Sep 27 16:41:18 2026	Iker Alejandro Arriaga López	Reparte las cartas inciales a los tres jugadores
c140c2a	Sun Sep 27 16:35:18 2026	Iker Alejandro Arriaga López	Modifica la interfaz para ajustarse mejor a los elementos utilizados
837a5e6	Sat Sep 26 19:57:19 2026	f3rhg	Update Program.cs
afa9390	Sat Sep 26 19:48:54 2026	Fernanda	Conexion a la base de datos
16d38e8	Sat Sep 26 17:09:48 2026	Iker Alejandro Arriaga López	Cambio de ubicacion de las clases Baraja y Carta

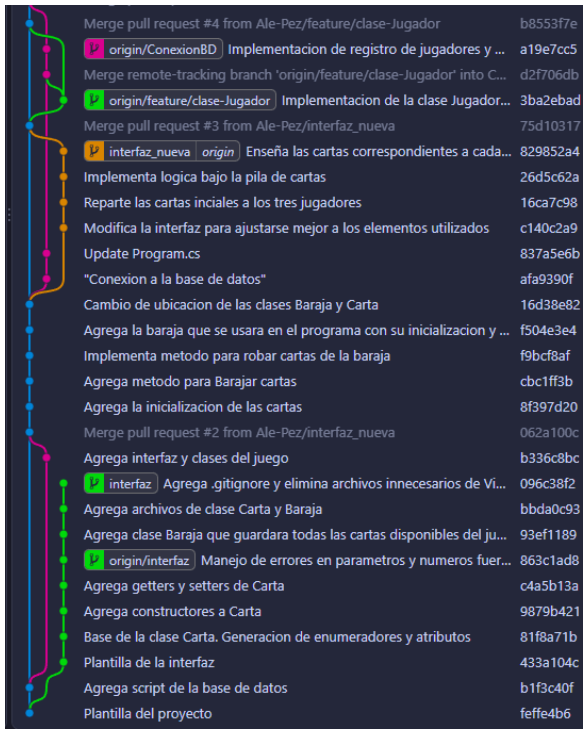
f504e3e	Sat Sep 26 17:08:31 2026	Iker Alejandro Arriaga López	Agrega la baraja que se usara en el programa con su inicializacion y reparticion de cartas (solo al jugador 1)
f9bcf8a	Sat Sep 26 16:39:43 2026	Iker Alejandro Arriaga López	Implementa metodo para robar cartas de la baraja
cbc1ff3	Sat Sep 26 16:35:49 2026	Iker Alejandro Arriaga López	Agrega metodo para Barajar cartas
8f397d2	Sat Sep 26 16:25:54 2026	Iker Alejandro Arriaga López	Agrega la inicializacion de las cartas
062a100	Sat Sep 26 15:57:21 2026	Iker Alejandro Arriaga López	Merge pull request #2 from Ale-Pez/interfaz_nueva - Agrega interfaz y clases del juego
b336c8b	Sat Sep 26 15:53:25 2026	Iker Alejandro Arriaga López	Agrega interfaz y clases del juego
096c38f	Sat Sep 26 15:24:58 2026	Iker Alejandro Arriaga López	Agrega .gitignore y elimina archivos innecesarios de Visual Studio
bbda0c9	Sat Sep 26 15:15:58 2026	Iker Alejandro Arriaga López	Agrega archivos de clase Carta y Baraja
93ef118	Sat Sep 26 15:14:46 2026	Iker Alejandro Arriaga López	Agrega clase Baraja que guardara todas las cartas disponibles del juego

863c1ad	Fri Sep 25 16:00:41 2026	Iker Alejandro Arriaga López	Manejo de errores en parametros y numeros fuera del rango
c4a5b13	Fri Sep 25 15:44:10 2026	Iker Alejandro Arriaga López	Agrega getters y setters de Carta
9879b42	Fri Sep 25 15:41:13 2026	Iker Alejandro Arriaga López	Agrega constructores a Carta
81f8a71	Fri Sep 25 15:37:53 2026	Iker Alejandro Arriaga López	Base de la clase Carta. Generacion de enumeradores y atributos
433a104	Fri Sep 25 15:25:06 2026	Iker Alejandro Arriaga López	Plantilla de la interfaz
b1f3c40	Wed Sep 23 16:07:44 2026	Regina	Agrega script de la base de datos
feffe4b	Tue Sep 22 11:18:51 2026	Iker Alejandro Arriaga López	Plantilla del proyecto

LISTADO DE RAMAS GENERADAS




mejoras_interfaz origin Soluciona error al cambiar de color con ... bea426be
Agrega una figura para indicar el color de carta del juego actual c328ad67
Merge pull request #16 from Ale-Pez/mejoras_interfaz 153416f5
Recoloca la etiqueta que emerge al jugar ciertas cartas (ej. +4) c30a7a65
Los controles del centro (Mazo, Pila, etc.) Han sido centrados. Tambi... f35e8b27
Merge pull request #15 from Ale-Pez/feature/CambiosMensajesSpam 442d602a
origin/feature/CambiosMensajesSpam Ignorar carpeta pycache e... ff0949e4
Modificacion mensajes Spam a Label con timer 810fac3a
Merge pull request #14 from Ale-Pez/API e6a35a51
Modificacion de la interfaz "Registro" 7958065a
Merge pull request #13 from Ale-Pez/feature/ReinicioJuego-Barajeo... d2e64861
origin/feature/ReinicioJuego-BarajeoPilaCartas Reinicio de juego e... ccc93e5c
Reinicio de juego en Form1 0eddbde2
Barajar pila de cartas ab844442
Merge pull request #12 from Ale-Pez/API 782743b2
Se adjuntó el archivo que contiene la API al prooyecto 00176b34
Merge pull request #11 from Ale-Pez/API 74a73309
Ajustes en la ventana de "resumen de partida" y conexion de la API ... 3f0d9848
Merge pull request #10 from Ale-Pez/feature/cambio-002-bd 21c2c392
origin/feature/cambio-002-bd Segunda actualizacion de la BD, el... b05e63a5
Merge pull request #9 from Ale-Pez/API 05938046
correciones en la logica de penalizacion 205d75fd
Implementacion de la penalizacion (si el jugadore no dice UNO ante... 7799c7b9
implementacion del metodo "verificar ganador" 292ca7af
Implementacion de la interfaz "resumen de partida" que obtiene el f... 6d69b090
Creacion de la clase apiService (puente de comunicación entre C# y ... b1102da7
Merge pull request #8 from Ale-Pez/feature/boton_uno 8870af29
feature/boton_uno origin Comprobación de que la primer carta... e9637b46
Agrega el boton de UNO y lo oculta al iniciar la ventana Form1 6c302127
Merge pull request #7 from Ale-Pez/feature/TiradaDeCartas-Agrega... 6986b43a
origin/feature/TiradaDeCartas-AgregarALogJuego Actualizar log-j... 6dbac510
Funcion para escoger color del comodin y robar del mazo restante 49804ea1
Evento tira carta, Funcion valida turno y Aplicar efecto de carta 850612d6
Verificar Jugada Valida y Logica de turnos: Avanzar y Actualizar 5b3f793a
Merge pull request #6 from Ale-Pez/feature/cambio-001-bd f89a99c0
origin/feature/cambio-001-bd Base de datos corregida en histori... 6c2cacdb
Merge pull request #5 from Ale-Pez/ConexionBD 30d638fb


Merge pull request #4 from Ale-Pez/feature/clase-Jugador b8553f7e
origin/ConexionBD Implementacion de registro de jugadores y ... a19e7cc5
Merge remote-tracking branch 'origin/feature/clase-Jugador' into C... d2f706db
origin/feature/clase-Jugador Implementacion de la clase Jugador... 3ba2ebad
Merge pull request #3 from Ale-Pez/interfaz_nueva 75d10317
interfaz_nueva origin Enseña las cartas correspondientes a cada... 829852a4
Implementa logica bajo la pila de cartas 26d5c62a
Reparte las cartas inciales a los tres jugadores 16ca7c98
Modifica la interfaz para ajustarse mejor a los elementos utilizados c140c2a9
Update Program.cs 837a5e6b
"Conexion a la base de datos" afa9390f
Cambio de ubicacion de las clases Baraja y Carta 16d38e82
Agrega la baraja que se usara en el programa con su inicializacion y ... f504e3e4
Implementa metodo para robar cartas de la baraja f9bcf8af
Agrega metodo para Barajar cartas cbc1ff3b
Agrega la inicializacion de las cartas 8f397d20
Merge pull request #2 from Ale-Pez/interfaz_nueva 062a100c
Agrega interfaz y clases del juego b336c8bc
interfaz Agrega .gitignore y elimina archivos innecesarios de Vi... 096c38f2
Agrega archivos de clase Carta y Baraja bbda0c93
Agrega clase Baraja que guardara todas las cartas disponibles del ju... 93ef1189
origin/interfaz Manejo de errores en parametros y numeros fuer... 863c1ad8
Agrega getters y setters de Carta c4a5b13a
Agrega constructores a Carta 9879b421
Base de la clase Carta. Generacion de enumeradores y atributos 81f8a71b
Plantilla de la interfaz 433a104c
Agrega script de la base de datos b1f3c40f
Plantilla del proyecto feffe4b6

RELACIÓN COMMITS-TAREAS

Regina:

Tarea	Commit(s)
Creación de la base de datos y diagrama entidad-relación.	**b1f3c40** Wed Sep 23 16:07:44 2026 Agrega script de la base de datos **6c2cacd** Sat Oct 3 11:44:53 2026 Base de datos corregida en historial_partida y log_juego **b05e63a** Sun Oct 4 12:38:57 2026 Segunda actualizacion de la BD, el juego ya guarda los datos en la BD
Creación de la clase Jugador con atributos y métodos.	**3ba2eba** Mon Sep 28 19:50:33 2026 Implementacion de la clase Jugador y actualizacion de Repartir_Cartas
Lógica de los turnos.	**5b3f793** Sat Oct 3 14:16:01 2026

	Verificar Jugada Valida y Logica de turnos: Avanzar y Actualizar
Lógica de las jugadas (Tirada de cartas).	**5b3f793** Sat Oct 3 14:16:01 2026 Verificar Jugada Valida y Logica de turnos: Avanzar y Actualizar **850612d** Sat Oct 3 16:02:14 2026 Evento tira carta, Funcion valida turno y Aplicar efecto de carta
Implementación de ventanas para caso COMODÍN.	**49804ea** Sat Oct 3 18:41:11 2026 Funcion para escoger color del comodin y robar del mazo restante
Implementación del robo desde mazo.	**49804ea** Sat Oct 3 18:41:11 2026 Funcion para escoger color del comodin y robar del mazo restante **6dbac51** Sat Oct 3 19:22:13 2026

	Actualizar log-juego y registrar-jugador **ab84444** Sun Oct 4 19:08:25 2026 Barajar pila de cartas
Reinicio Juego.	**0eddbde** Sun Oct 4 20:00:34 2026 Reinicio de juego en Form1 **ccc93e5** Sun Oct 4 20:31:12 2026 Reinicio de juego en Form3
Implementación de la clase Cliente Juego(1ra versión)	**515fb22** Fri Oct 30 11:58:38 2026 Implementación de la clase Cliente Juego
Hacer la conexión básica con el Servidor	**65fb9ba** Wed Nov 4 08:30:02 2026 Conexión al Servidor/ Envio de Acción de Carta
Paso de lógica del juego al Servidor	**2563d0d** Wed Nov 4 09:42:47 2026

	Cambio de logica del juego de LogicaUno a ServidorUno
Modificar funciones de Lógica del juego	**62893e1** Wed Nov 4 09:57:04 2026 Modificaciones de MesaUno

Iker:

Tarea	Commit(s)
Implementación de la interfaz de juego.	**433a104** Fri Sep 25 15:25:06 2026 Plantilla de la interfaz **b336c8b** Sat Sep 26 15:53:25 2026 Agrega interfaz y clases del juego **f504e3e** Sat Sep 26 17:08:31 2026 Agrega la baraja que se usara en el programa con su inicializacion y reparticion de cartas (solo al jugador 1) **c140c2a** Sun Sep 27 16:35:18 2026 Modifica la interfaz para ajustarse mejor a los elementos utilizados

	16ca7c9 Sun Sep 27 16:41:18 2026 Reparte las cartas inciales a los tres jugadores **26d5c62** Sun Sep 27 16:54:30 2026 Implementa logica bajo la pila de cartas **829852a** Sun Sep 27 17:41:38 2026 Enseña las cartas correspondientes a cada jugador y la pila **6c30212** Sat Oct 3 20:19:26 2026 Agrega el boton de UNO y lo oculta al iniciar la ventana Form1 **e9637b4** Sat Oct 3 20:25:31 2026 Comprobación de que la primer carta de la pila sea únicamente numérica **f35e8b2** Thu Oct 8 11:49:40 2026 Los controles del centro (Mazo, Pila, etc.) Han sido centrados. Tambien se han hecho ligeras correcciones de ortografía o código muerto

	c30a7a6 Sat Oct 10 12:43:29 2026 Recoloca la etiqueta que emerge al jugar ciertas cartas (ej. +4) **c328ad6** Sat Oct 10 18:08:14 2026 Agrega una figura para indicar el color de carta del juego actual **bea426b** Sat Oct 10 18:26:18 2026 Soluciona error al cambiar de color con una carta numerica. Quita la figura de referencia para la figura de colores
Creación de la clase Carta:	**81f8a71** Fri Sep 25 15:37:53 2026 Base de la clase Carta. Generacion de enumeradores y atributos **9879b42** Fri Sep 25 15:41:13 2026 Agrega constructores a Carta **c4a5b13** Fri Sep 25 15:44:10 2026

	Agrega getters y setters de Carta **863c1ad** Fri Sep 25 16:00:41 2026 Manejo de errores en parametros y numeros fuera del rango **bbda0c9** Sat Sep 26 15:15:58 2026 Agrega archivos de clase Carta y Baraja *(A la solución)* **b336c8b** Sat Sep 26 15:53:25 2026 Agrega interfaz y clases del juego **16d38e8** Sat Sep 26 17:09:48 2026 Cambio de ubicacion de las clases Baraja y Carta
Creación de la clase Baraja.	**93ef118** Sat Sep 26 15:14:46 2026 Agrega clase Baraja que guardara todas las cartas disponibles del juego **bbda0c9** Sat Sep 26 15:15:58 2026 Agrega archivos de clase Carta y Baraja *(A la solución)*

	b336c8b Sat Sep 26 15:53:25 2026 Agrega interfaz y clases del juego **8f397d2** Sat Sep 26 16:25:54 2026 Agrega la inicializacion de las cartas **cbc1ff3** Sat Sep 26 16:35:49 2026 Agrega metodo para Barajar cartas **16d38e8** Sat Sep 26 17:09:48 2026 Cambio de ubicacion de las clases Baraja y Carta **f9bcf8a** Sat Sep 26 16:39:43 2026 Implementa metodo para robar cartas de la baraja **f504e3e** Sat Sep 26 17:08:31 2026 Agrega la baraja que se usara en el programa con su inicializacion y reparticion de cartas (solo al jugador 1)

Separación de responsabilidades de Form1	**bd8bdb7** Thu Oct 15 17:08:59 2026 Primer avance de la reestructuración del código: Separa de Form1 la logica del juego, las llamadas a la API y el manejo de la interfaz; cada uno en su propia clase (LogicaUNO, LlamadaBD, MenuUNO respectivamente) **a4d8ca4** Thu Oct 15 17:10:28 2026 Agrega las clases de reestructuración del código: LlamadaBD, LogicaUNO y MesaUNO **da1d5bd** Sun Oct 18 17:27:40 2026 Separación de responsabilidades completa, MesaUNO se encarga de la interfaz gráfica, LogicaUNO de aplicar los cambios internos del juego y LlamadaBD de conectarse con la API

Fernanda:

Tarea	Commit(s)
Conexión inicial a la base de datos.	**afa9390** Sat Sep 26 19:48:54 2026 Conexion a la base de datos **837a5e6** Sat Sep 26 19:57:19 2026

	Update Program.cs **3f0d984** Sun Oct 4 14:27:55 2026 Ajustes en la ventana de "resumen de partida" y conexion de la API a c# **7687fe4** Sat Oct 10 21:51:07 2026 cambios en forms3.cs
Implementación de registro de jugadores y conexión a la base de datos.	**A19e7cc** Wed Sep 30 17:29:31 2026 Implementacion de registro de jugadores y base de datos **7958065** Tue Oct 6 11:53:11 2026 Modificacion de la interfaz "Registro"
Implementación de la API.	**b1102da** Sat Oct 3 21:41:49 2026 Creacion de la clase apiService (puente de comunicación entre C# y la api) **00176b3**

	Sun Oct 4 14:37:14 2026 Se adjuntó el archivo que contiene la API al proyecto **7687fe4** Sat Oct 10 21:51:07 2026 cambios en forms3.cs **86f28c8** Sat Oct 10 22:18:44 2026 Se agrega script iniciar_api.bat para automatizar la API
Implementación de la ventana “Resumen de partida”.	**6d69b09** Sat Oct 3 22:52:46 2026 Implementacion de la interfaz "resumen de partida" que obtiene el formato json de la API **3f0d984** Sun Oct 4 14:27:55 2026 Ajustes en la ventana de "resumen de partida" y conexion de la API a c#
Implementación de la lógica para verificar que un jugador haya ganado.	**292ca7a** Sat Oct 3 23:31:02 2026 implementacion del metodo "verificar ganador"

Implementación la lógica para la penalización.	**7799c7b** Sun Oct 4 02:44:28 2026 Implementacion de la penalizacion (si el jugador no dice UNO antes de tirar su última carta se agregan dos cartas a su panel) **205d75f** Sun Oct 4 03:15:59 2026 correciones en la logica de penalizacion

PROBLEMAS ENCONTRADOS Y SOLUCIONES

Descripción del problema	Solución
Iker: Al querer hacer merge desde una rama llamada “interfaz” a main GitHub no podía realizar la combinación debido a un desfaz de versiones del main en linea y remoto.	Cree otra rama generada desde el main actualizado y manualmente copie los cambios hechos en mi rama en la nueva.
Iker: El único detalle con la base de datos fue que, como fue creada por mi compañera Regina, tuve que obtener el script y configurarla de forma local; aunque este proceso fue un poco tedioso, era totalmente necesario para comprobar que la conexión y el registro se realizaran con éxito.	Para resolverlo, tomé el script, y lo ejecuté paso a paso en MySQL para estructurar correctamente las tablas correspondientes, esto se tuvo que hacer varias veces porque no funcionaba.

Fernanda: Tanto Regina como yo no nos veíamos reflejadas como contribuidores del proyecto en GitHub, a pesar de que se vean reflejados nuestros cambios y commits.	Esto fue un problema que como tal no pudimos solucionar, debido a que creamos usuarios “ficticios” en una de las prácticas en clase. Nuestros usuarios no existen como cuentas reales en GitHub, por lo que, si aparecen los cambios, pero GitHub no nos reconoce como usuarios de su plataforma. La solución fue volver a declarar nuestro nombre y correo en Git, pero eso solo soluciono que los cambios futuros salieran a nuestra cuenta real. Por suerte, nos dimos cuenta antes de realizar cambios fuertes, por lo que las métricas de GitHub son bastante acertadas al trabajo final.
Regina: Problemas al intentar hacer git pull origin main o git checkout main, el problema se debía a archivos generados localmente dentro de la carpeta __pycache__ mientras se ejecutaba la API.	Para resolver el problema se agregó los archivos que se querían ignorar al archivo .gitignore
Iker: Tuve problemas para conectarme al servidor en la primera prueba que hicimos. Mis otras dos compañeras si podian unirse a la partida, pero a mí no me dejaba entrar.	Nos dimos cuenta de que yo me encontraba en una subred distinta a la de mis compañeras. A pesar de estar en la misma red, el servidor no aceptaba mi host debido a esa razon. La solución, intentar estar en la misma subred. No es una solución en si, pero si se tiene cuidado en eso, funciona excelente.

DESCRIPCIÓN DE LOS CAMBIOS

- ## Objetivo de la nueva implementación.

El proyecto original de UNO, desarrollado en C# Windows Forms, se adaptó para permitir que tres jugadores participaran en una misma partida desde computadoras diferentes. Para lograrlo, se implementó un servidor TCP central encargado de coordinar el desarrollo del juego, controlar los turnos y sincronizar las acciones de los jugadores. Además, se modificó la API desarrollada con FastAPI para centralizar el registro de jugadores, partidas, movimientos y resultados en una sola base de datos MySQL, a diferencia de la implementación anterior, en la que cada computadora almacenaba la información en su propia base de datos. De esta manera, se logró que los tres jugadores compartieran tanto el estado de la partida como la información registrada.

- ## Comunicación entre el servidor y el cliente UNO

Para permitir que los tres jugadores participaran en una misma partida desde diferentes computadoras, fue necesario implementar una comunicación mediante sockets TCP. Esta comunicación se realiza principalmente a través de las clases **ClienteJuego**, **MesaUNO** y **LogicaUNO**, mientras que **ServidorUNO** se encarga de recibir las acciones, validarlas y enviar las actualizaciones correspondientes.

ClienteJuego.cs

Esta clase funciona como intermediaria entre la interfaz gráfica y el servidor. Para ello, se implementaron y adaptaron las siguientes funciones:

- **ConectarAlServidor():** Establece la conexión TCP con el servidor utilizando su dirección IP y el puerto 11111. Una vez conectado, inicia un hilo encargado de escuchar los mensajes enviados por el servidor.
- **EscucharAlServidor():** Permite recibir continuamente los mensajes del servidor sin detener la ejecución de la interfaz gráfica. Los mensajes se procesan utilizando saltos de línea para identificar cada instrucción recibida.
- **ProcesarOrdenDelServidor():** Interpreta los mensajes enviados por el servidor, como la asignación del número de jugador, el reparto inicial de cartas, la carta central y el turno actual. Posteriormente, comunica la información a `MesaUNO` mediante el evento `MensajeRecibido`.

- **EnviarAccion():** Envía al servidor las acciones realizadas por el jugador, como tirar una carta, robar del mazo, presionar el botón UNO o solicitar el reinicio de la partida.
- **Desconectar():** Cierra la conexión TCP cuando el jugador sale del juego.

MesaUNO.cs

Se modificó para que la interfaz gráfica mostrara la información recibida del servidor, en lugar de controlar la partida de manera independiente.

- **ProcesarMensajeServidor():** Recibe los mensajes transmitidos por **ClienteJuego** e identifica la acción correspondiente. Dependiendo del mensaje, actualiza la mano del jugador, la carta central, el turno, el sentido del juego, las cantidades de cartas o el ganador.
- **ActualizarInterfaz():** Actualiza los elementos gráficos del tablero de acuerdo con el estado de la partida. Se optimizó para evitar que los controles se reconstruyeran constantemente después de cada jugada.
- **DibujarMano():** Muestra las cartas del jugador local y los reversos de las cartas de los oponentes. Se incorporó una comparación del estado visual para redibujar únicamente cuando fuera necesario.

Además, se utilizó **BeginInvoke** para ejecutar las actualizaciones de los controles desde el hilo de la interfaz, ya que los mensajes del servidor llegan mediante un hilo secundario.

También se modificó el procesamiento del mensaje **REINICIO_CONFIRMADO**, permitiendo limpiar el estado de la ronda anterior y esperar las nuevas cartas y turnos sin cerrar las ventanas ni perder la conexión con el servidor.

LogicaUNO.cs

Esta clase se adaptó para mantener una representación local de la partida, utilizando la información enviada por el servidor.

Entre las funciones implementadas o modificadas se encuentran:

- **RecibirNombresServidor():** Actualiza los nombres de los tres jugadores con la información enviada por el servidor.
- **RecibirManoServidor():** Recibe las cartas iniciales correspondientes al jugador local y actualiza su mano.
- **RecibirCartaMesaServidor():** Actualiza la carta que se encuentra en el centro del tablero.

- **ActualizarTurnoServidor():** Sincroniza el turno local con el turno establecido por el servidor, evitando que los clientes determinen independientemente a quién le corresponde jugar.
- **ActualizarColorServidor():** Actualiza el color activo de la partida, especialmente después de jugar un comodín o una carta +4.
- **ConfirmarJugadaServidor():** Refleja en la lógica local una jugada que ya fue validada y confirmada por el servidor.
- **ReiniciarRonda():** Restablece las variables de la partida, como las manos, los turnos, el sentido del juego y el estado del ganador, permitiendo iniciar una nueva ronda sin cerrar la aplicación.

De esta manera, **LogicaUNO** dejó de ser la clase encargada de validar las acciones y determinar los turnos, ya que estas responsabilidades se trasladaron al servidor.

Funciones implementadas en ServidorUNO

El archivo Program.cs del servidor se modificó para centralizar la lógica del juego y permitir la comunicación simultánea con los tres clientes.

Las principales funciones implementadas o adaptadas fueron:

- **IniciarServidor():** Crea el socket TCP, establece el puerto 11111 y espera la conexión de los tres jugadores. A cada conexión le asigna un número de jugador.

- **EscucharCliente():** Ejecuta un hilo para recibir los mensajes de cada jugador, permitiendo que el servidor atienda varias conexiones simultáneamente.

- **ProcesarMensaje():** Interpreta las instrucciones recibidas, como CONECTAR, JUGAR_CARTA, ROBAR_CARTA, DECISION_ROBO, UNO y REINICIAR, y ejecuta la operación correspondiente.

- **IniciarPartida():** Prepara la baraja, reparte siete cartas a cada jugador, selecciona la carta inicial y establece el primer turno. Posteriormente, envía esta información a los tres clientes.

- **JugadaValida():** Comprueba que la carta seleccionada pueda jugarse de acuerdo con las reglas implementadas, considerando el color, número y tipo de carta.

- **JugarCarta():** Procesa una jugada válida, retira la carta de la mano, actualiza la carta central y aplica los efectos correspondientes, como bloqueo, reversa, +2 o +4.

- **EntregarCartas():** Agrega cartas a la mano de un jugador y comunica los cambios a los clientes.

- **TerminarTurno():** Determina el siguiente jugador de acuerdo con el sentido de la partida y envía el nuevo turno.

- **EnviarJugador():** Envía un mensaje a un jugador específico, por ejemplo, para comunicarle las cartas que recibió.

- **EnviarATodos():** Transmite un mensaje a los tres clientes conectados para mantener sincronizados sus tableros.

- **CrearRegistroPartida():** Se comunica con la API para registrar a los jugadores y crear una partida compartida en la base de datos.

- **GuardarLog():** Registra los movimientos de los jugadores mediante peticiones a la API.

- **RegistrarGanador():** Actualiza el historial de victorias y derrotas y registra la finalización de la partida.

Modificaciones realizadas en la API y Base de Datos

Motivo de las modificaciones

Inicialmente, el proyecto UNO estaba diseñado para ejecutarse de manera local, por lo que cada computadora realizaba sus propias operaciones de registro y almacenamiento de información en una base de datos independiente. Esto representaba un problema al implementar el modo multijugador, ya que los tres jugadores debían participar en una misma partida y compartir información como los movimientos realizados, el ganador y las estadísticas de cada usuario.

Por esta razón, fue necesario modificar la comunicación con la API, de manera que las tres computadoras pudieran acceder a una misma base de datos MySQL, ubicada en la computadora principal que actúa como servidor.

Modificaciones en la comunicación con la API

Anteriormente, la API se ejecutaba utilizando la dirección `127.0.0.1`, lo que permitía acceder a ella únicamente desde la computadora donde estaba ejecutándose. Sin embargo, al implementar la comunicación entre tres computadoras, fue necesario permitir que los clientes secundarios también pudieran consultar la información almacenada.

Para solucionar este problema, se modificó el archivo `.bat` encargado de iniciar la API, configurando Uvicorn de la siguiente manera:

start uvicorn conexionApi:app --host 0.0.0.0 --port 8000 --reload

El parámetro --host 0.0.0.0 permite que la API escuche las conexiones recibidas a través de las interfaces de red de la computadora principal, mientras que --port 8000 establece el puerto utilizado para la comunicación HTTP.

También fue necesario modificar las direcciones IP utilizadas por los clientes en los archivos Registro.cs, LlamadaBD.cs y en las consultas del formulario Form3.cs, para que las peticiones se dirigieran a la computadora principal y no a la dirección local de cada cliente.

Centralización del registro de partidas

Otra modificación importante consistió en trasladar al servidor la responsabilidad de registrar y finalizar las partidas.

En la implementación anterior, cada instancia del juego podía realizar sus propias operaciones sobre la base de datos. Esto provocaba que se generaran registros independientes, aunque los jugadores estuvieran participando en una misma partida.

Para evitarlo, se modificó el funcionamiento del sistema de manera que ServidorUNO fuera el encargado de comunicarse con la API para crear una única partida compartida.

Para ello, se implementaron las siguientes funciones en ServidorUNO:

- **PostApi():** Realiza las peticiones HTTP POST desde el servidor TCP hacia FastAPI, enviando los datos correspondientes en formato JSON.

- **CrearRegistroPartida():** Registra o recupera los identificadores de los jugadores y crea una nueva partida en la base de datos. El identificador generado se comparte con los clientes mediante el mensaje ID_PARTIDA.

- **GuardarLog():** Envía a la API los movimientos realizados por cada jugador, asociándolos con el identificador de la partida y del participante correspondiente.

- **RegistrarGanador():** Registra al ganador, actualiza las estadísticas de victorias y derrotas y cambia el estado de la partida a finalizada.

De esta manera, se evita que cada computadora cree una partida diferente y se garantiza que los movimientos y resultados correspondan a una misma sesión de juego.

Endpoints utilizados

Se utilizaron los siguientes endpoints:

- **POST /jugadores/registrar:** Registra a un jugador nuevo o recupera su identificador si ya existe en la base de datos.
- **POST /partidas/crear:** Crea una nueva partida y devuelve su identificador.
- **POST /logs:** Almacena los movimientos realizados por los jugadores durante la partida.
- **POST /historial/actualizar:** Actualiza las estadísticas de partidas ganadas y perdidas.
- **POST /partidas/finalizar:** Registra al ganador y marca la partida como finalizada.
- **GET /jugadores:** Recupera la información de los jugadores y sus estadísticas para mostrar el ranking global en el formulario Form3.

Funcionamiento de la comunicación

Cuando un jugador realiza una acción en MesaUNO, esta se envía mediante ClienteJuego al servidor TCP. El servidor recibe el mensaje, verifica que la acción sea válida y actualiza el estado general de la partida.

Posteriormente, el servidor comunica los cambios a los clientes conectados. ClienteJuego recibe los mensajes y los transmite a MesaUNO, donde se actualiza el estado local mediante LogicaUNO y se reflejan los cambios en la interfaz gráfica.

Gracias a esta implementación, los tres jugadores pueden visualizar una misma partida sincronizada, respetando los turnos establecidos por el servidor y manteniendo la información de las jugadas actualizada en las tres computadoras.